

Early Stopping for Large Reasoning Models via Confidence Dynamics

Parsa Hosseini^{1*} Sumit Nawathe^{1*} Mahdi Salmani^{2*}
 Meisam Razaviyayn² Soheil Feizi¹

¹University of Maryland ²University of Southern California

Abstract

Large reasoning models rely on long chain-of-thought generation to solve complex problems, but extended reasoning often incurs substantial computational cost and can even degrade performance due to overthinking. A key challenge is determining when the model should stop reasoning and produce the final answer. In this work, we study the confidence of intermediate answers during reasoning and observe two characteristic behaviors: correct reasoning trajectories often reach high-confidence answers early, while incorrect rollouts tend to produce long, unproductive reasoning traces and exhibit less reliable confidence dynamics. Motivated by these observations, we propose CoDE-Stop (Confidence Dynamics Early Stop), an early stopping method that leverages the dynamics of intermediate answer confidence to decide when to terminate reasoning, requiring no additional training and easily integrating into existing models. We evaluate CoDE-Stop on diverse reasoning and science benchmarks across multiple models. Compared to prior early stopping methods, it achieves a more favorable accuracy–compute tradeoff and reduces total token usage by 25–50% compared to standard full-length reasoning. In addition, we provide analyses of confidence dynamics during reasoning, offering insights into how confidence changes in both correct and incorrect trajectories.

1 Introduction

Generating long chain-of-thought (CoT) enables large language models (LLMs) to solve complex problems, but at a substantial computational cost. As reasoning models are increasingly deployed with long generation budgets (often spanning thousands of tokens) efficient inference becomes critical. In particular, reducing unnecessary reasoning steps is essential for improving the practicality and scalability of these systems. A key challenge is determining when to terminate reasoning and produce the final answer.

Several lines of work have sought to improve the efficiency of reasoning in LLMs. Some approaches rely on training-time modifications, such as compressing CoT (Arora & Zanette, 2025) or training auxiliary models (Zhang et al., 2025) for early stopping, but these require additional training and are not directly applicable at inference time. More recent work focuses on inference-time strategies that terminate reasoning by monitoring signals during generation, typically via intermediate answer predictions (Yang et al., 2025; Liu & Wang, 2025; Wei et al., 2026; Yong et al., 2025). These methods are effective at stopping trajectories that have already reached a reliable answer, but are less effective when reasoning becomes unproductive (such as when the model enters loops or exhibits unstable progress) often resulting in unnecessarily long generations.

To better understand when reasoning can be terminated, we analyze how model confidence evolves over the course of generation. We observe that reasoning trajectories exhibit distinct behaviors depending on their outcome. For trajectories that lead to correct answers, confidence typically increases over time and reaches a high value well before generation ends. In

*Equal Contribution. Code is available at <https://github.com/sudoparsa/CoDE-Stop>

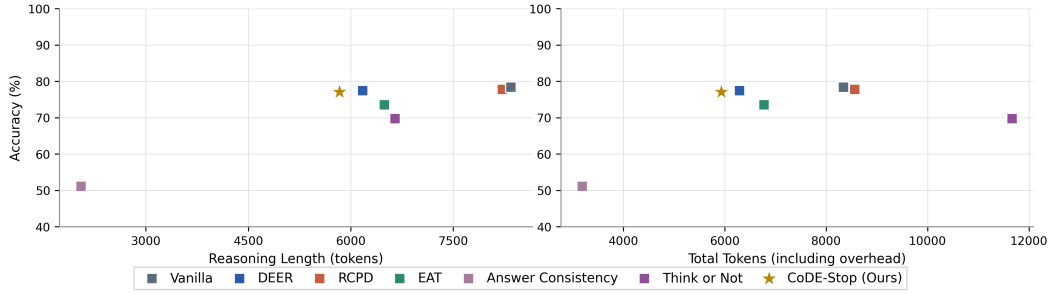


Figure 1: Accuracy vs. compute cost on Qwen3-4B averaged over 4 reasoning and science benchmarks. **Left:** reasoning length only. **Right:** total token compute including intermediate answer-generation overhead. CoDE-Stop achieves a stronger accuracy–compute tradeoff among early stopping methods.

contrast, incorrect trajectories often exhibit unstable behavior, with confidence fluctuating. Motivated by these observations, we propose CoDE-Stop (Confidence Dynamics Early Stopping), an inference-time method that leverages the temporal behavior of confidence to decide when to terminate reasoning. CoDE-Stop monitors confidence over reasoning steps and combines two complementary signals: a threshold on confidence to identify when the model has reached a reliable answer, and a degeneration signal to detect unproductive reasoning characterized by unstable or non-improving confidence.

We evaluate CoDE-Stop on various models across reasoning and science datasets, comparing against prior inference-time early stopping methods that rely on intermediate answer generation. Figure 1 shows the tradeoff between accuracy and token usage. Whether measuring reasoning length alone or total token compute including intermediate answer overhead, CoDE-Stop achieves a more favorable accuracy–compute tradeoff, reducing token usage by 25% while maintaining accuracy comparable to strong baselines.

Beyond overall performance, our analysis reveals a key property of confidence dynamics: the early stages of reasoning provide more informative signals for distinguishing correct and incorrect trajectories. At the same time, models tend to become increasingly confident even along incorrect trajectories, making late-stage confidence unreliable for detecting failure. These observations suggest that confidence should not be treated uniformly over time. They directly inform our design: CoDE-Stop combines a degeneration signal with non-uniform weighting over reasoning steps. Together, our results demonstrate that CoDE-Stop is a simple, training-free, and effective approach for improving the efficiency of reasoning in LLMs.

2 Confidence Dynamics in Reasoning Models

2.1 Problem Setup and Motivation

We consider a reasoning trajectory generated by an LLM as a sequence of tokens $x_1, x_2, \dots, x_T$. During generation, we identify a set of reasoning steps indexed by $i = 1, \dots, K$, where T_i denotes the token position of the i -th step. In practice, these steps correspond to specific transition points in the generation (e.g., tokens such as “Wait” or “\n\n”), following prior work (Yang et al., 2025; Wang et al., 2025).

We associate each reasoning step with a confidence score. At each step i , we prompt ((\n**Final Answer**\n\n The final answer is \boxed{...})) the model to produce an answer and compute a confidence score c_i as the average probability assigned to the generated answer tokens. This results in a sequence of confidence values $\{c_i\}_{i=1}^K$, capturing the evolution of the model’s belief throughout the reasoning process.

Our goal is to determine a stopping time t based on $\{c_i\}$, such that reasoning can terminate early without sacrificing answer correctness. In this section, we analyze the temporal

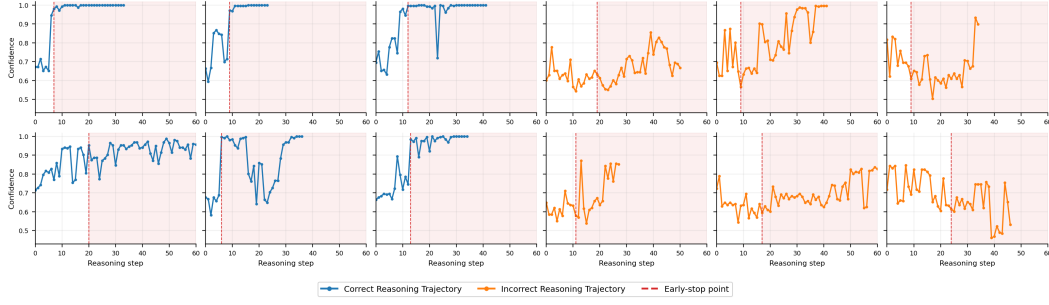


Figure 2: **Confidence dynamics across reasoning trajectories.** Correct trajectories reach high confidence early, while incorrect trajectories exhibit unstable and fluctuating confidence.

dynamics of confidence and show that correct and incorrect trajectories exhibit distinct patterns that can be leveraged for early stopping.

We first examine trajectories that lead to correct answers. Figure 2 shows representative examples when prompting Qwen3-4B (Team, 2025) on AIME24,25. As generation progresses, the confidence of correct trajectories increases rapidly and reaches a high value well before the end of the sequence. Despite this, the model often continues generating tokens after high confidence is already achieved, indicating that many later reasoning steps are unnecessary. This suggests that confidence provides a reliable signal for early stopping on successful trajectories, consistent with prior work such as Yang et al. (2025).

We next examine trajectories that lead to incorrect answers. We find that correct trajectories require on average 12K tokens, whereas incorrect trajectories extend to more than 25K tokens, doubling the total computation. Figure 3 shows that a similar pattern holds in terms of reasoning steps: incorrect trajectories are substantially longer and exhibit a heavy-tailed distribution, while correct trajectories typically terminate much earlier. This indicates that a significant portion of computation is dominated by long, unproductive reasoning trajectories that do not lead to correct answers.

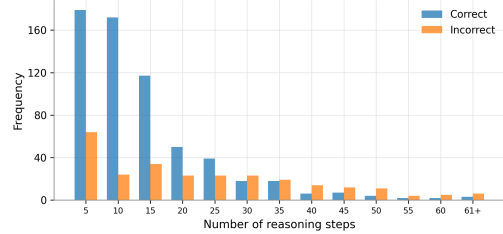


Figure 3: Incorrect trajectories are longer and exhibit a heavy-tailed distribution.

A key difficulty in stopping such trajectories is that confidence at a single step is not sufficient to determine whether reasoning should terminate. While high confidence indicates that the model has likely reached a final answer, low confidence is ambiguous: it may reflect exploration, after which the model can recover and arrive at a correct solution. However, incorrect trajectories often exhibit persistent instability, with confidence fluctuating over time without consistent improvement (Figure 2). This suggests that identifying unproductive reasoning requires reasoning over the full sequence of confidence values $\{c_i\}$, rather than relying on individual steps. In particular, these signals become more reliable when accumulated over multiple steps.

To further understand which parts of the trajectory are most informative, we analyze confidence across reasoning steps. Figure 4 (left) shows that early-stage confidence provides clearer separation between correct and incorrect trajectories, whereas later steps exhibit increased overlap. Notably, confidence tends to increase over time even for incorrect trajectories, making late-stage confidence unreliable for detecting failure.

Together, these observations suggest that effective early stopping should account for the temporal dynamics of confidence. In particular, identifying unproductive reasoning requires accumulating signs of instability over time, while placing greater emphasis on earlier reasoning steps, which provide more reliable signals. To capture these properties, we

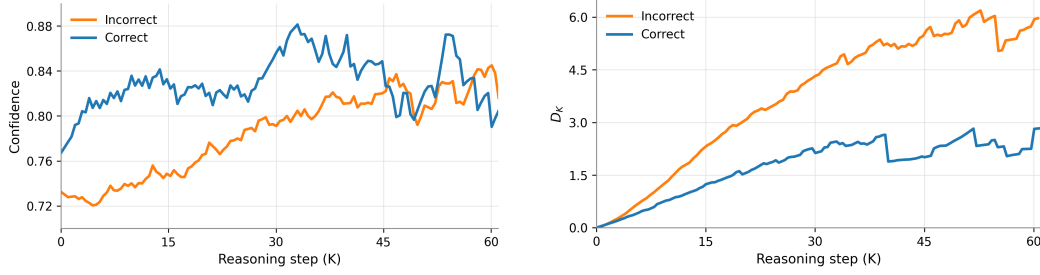


Figure 4: Confidence and degeneration dynamics over reasoning steps. **Left:** Average confidence for correct and incorrect rollouts; confidence increases even for incorrect cases, while early steps provide better separation. **Right:** Average degeneration score D_K ; incorrect rollouts consistently exhibit higher values, with the gap increasing over time.

introduce a degeneration score at step k that accumulates instability across reasoning steps:

$$D_k = \sum_{i=1}^k w_i v_i, \quad (1)$$

where v_i is an indicator of instability at step i , and w_i assigns higher importance to earlier steps in the trajectory. Intuitively, v_i captures whether confidence exhibits unstable behavior at a given step, while w_i emphasizes early-stage signals, which are more informative for distinguishing correct and incorrect trajectories.

Figure 4 (right) shows that this accumulated signal separates correct and incorrect trajectories early in the reasoning process, with incorrect trajectories consistently exhibiting higher degeneration scores. This suggests that the degeneration score provides a reliable criterion for identifying unproductive reasoning. In the next section, we provide a detailed formulation of v_i and w_i , and describe how the degeneration score can be used to enable effective early stopping.

2.2 Method

We propose **CoDE-Stop**, an early stopping method that leverages the temporal dynamics of confidence during reasoning. Our approach combines two complementary signals: (1) a confidence-based criterion to identify when the model has reached a reliable answer, and (2) a degeneration score that captures persistent instability in reasoning trajectories. Together, these signals enable early termination of both successful and unproductive reasoning.

At each reasoning step k , we determine whether to stop generation based on these two signals. We define a ramping confidence threshold

$$r_k = \min \left(r_{\max}, r_{\min} + \frac{r_{\max} - r_{\min}}{\text{steps}} \cdot k \right), \quad (2)$$

where $r_{\min}$, $r_{\max}$, and steps are constants. The parameter steps controls how quickly the threshold increases, reaching $r_{\max}$ after the specified number of reasoning steps and remaining constant thereafter. This reflects that higher confidence is required at later stages of reasoning.

We build on the degeneration score defined in Eq. 1 and specify the forms of the instability indicator v_i and weighting function w_i . We implement the instability indicator as

$$v_i = \mathbb{1}(2c_i - c_{i-1} < \delta), \quad (3)$$

where δ is a fixed threshold (set to 0.55 in all experiments). This captures whether confidence is low and fails to improve relative to previous step, indicating a lack of meaningful progress. The weighting function is defined as

$$w_i = \log \left(\frac{T_k}{T_i} \right) + 1, \quad (4)$$

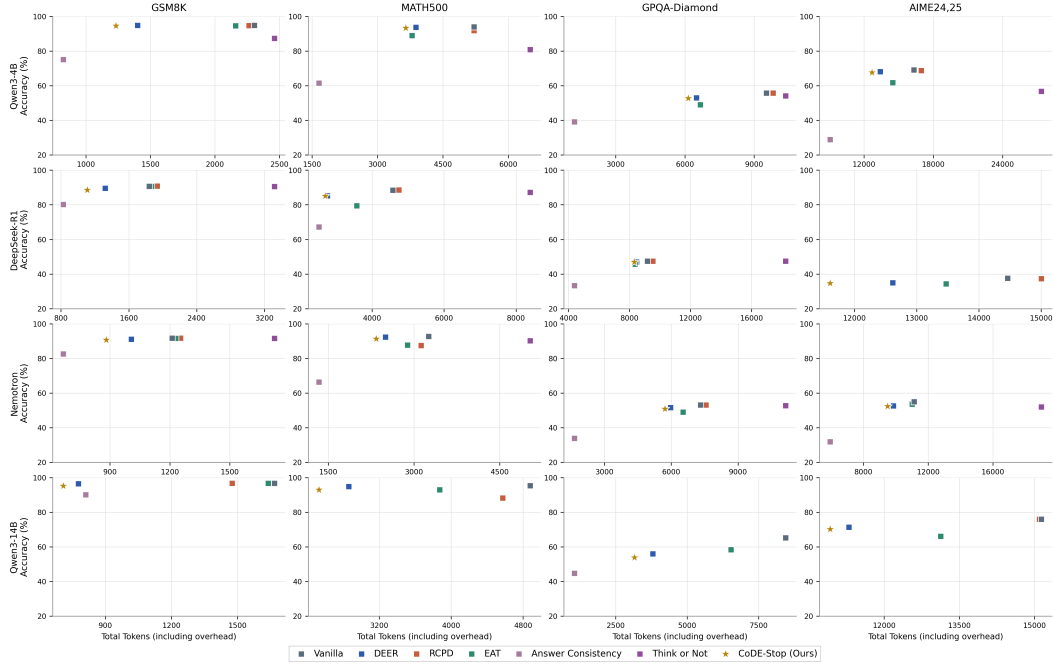


Figure 5: **Performance of CoDE-Stop against different baselines across multiple benchmarks.** CoDE-Stop consistently reduces inference cost while maintaining comparable performance to the baselines, making it Pareto optimal.

which assigns higher importance to earlier reasoning steps, consistent with our observation that early-stage signals are more informative. The logarithmic form provides a smooth growth over time while ensuring that the degeneration score increases monotonically with k . Further analysis of these design choices is provided in Section 4.2.

We stop generation at step k if either $c_k \geq r_k$ or $D_k \geq \tau$, where D_k is the degeneration score accumulated up to step k , and τ is a fixed threshold. The first condition captures trajectories that have reached sufficiently high confidence, while the second condition identifies trajectories with persistent instability. Once a stopping condition is met, we prompt the model using the same answer-generation prompt to produce the final answer. Both r_k and τ control the accuracy–compute tradeoff: lower values lead to earlier termination with reduced computation, while higher values allow longer reasoning and recover accuracy closer to the base model.

3 Experiments

3.1 Setup

Models. We evaluate CoDE-Stop on strong and diverse open-source reasoning models: Qwen3-4B,14B (Team, 2025), DeepSeek-R1-Distill-Llama-8B (DeepSeek-AI, 2025), and Llama-3.1-Nemotron-Nano-8B-v1 (et al., 2025), all accessed via the HuggingFace. For all models in our main experiments, we set the `max_new_tokens` to 32K. We provide the full generation and inference configurations for each model in the appendix.

Benchmarks. We evaluate CoDE-Stop on a diverse set of reasoning and science benchmarks: AIME 2024/2025 (60 questions) (2024,2025), MATH500 (500) (Hendrycks et al., 2021), GSM8K (1,319) (Cobbe et al., 2021), and GPQA-Diamond (198) (Rein et al., 2024).

Prompting Baselines. Prior work has explored improving reasoning efficiency through prompting strategies. These approaches are orthogonal to CoDE-Stop and can be combined with our method. We evaluate four prompting-based baselines, including Vanilla prompting,

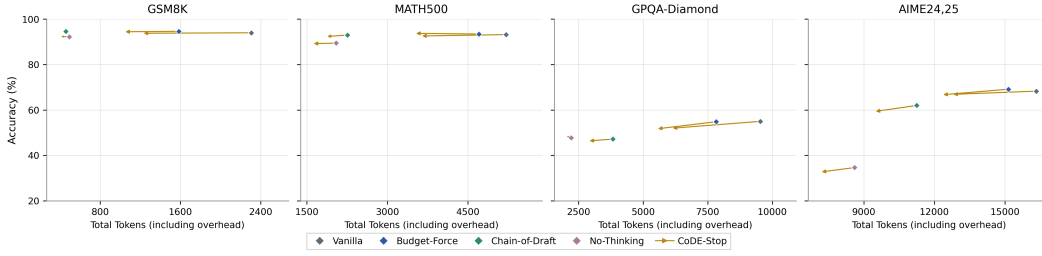


Figure 6: **Performance of CoDE-Stop with different prompting baselines.** CoDE-Stop can be combined with various prompting strategies to further improve performance.

Budget Forcing, Chain-of-Draft (CoD) (Xu et al., 2025a), and NoThinking (Ma et al., 2025). We provide the exact prompts used for each method in the Appendix A.

Evaluation Setup. For each prompt, we generate multiple reasoning trajectories per question: 15 for AIME, 2 for MATH500, 1 for GSM8K, and 5 for GPQA-Diamond, yielding about 1,000 trajectories per benchmark. Then, to reduce generation variance effect and run efficiently, we use these trajectories in all the baselines.

Baselines. We compare CoDE-Stop with several inference-time early stopping methods that rely on intermediate answer generation. As a reference, we also include a Vanilla setting, where the model is prompted normally without early stopping; to ensure a fair comparison, we force the model to produce a final answer when it reaches the maximum token budget, consistent with other baselines that explicitly trigger answer generation. We consider the following baselines: Think or Not (Yong et al., 2025), DEER (Yang et al., 2025), EAT (Wang et al., 2025), RCPD (Wei et al., 2026), and Answer Convergence (Liu & Wang, 2025).

Hyperparameters. All methods considered in Table 1 have hyperparameters that control the accuracy–compute tradeoff. For baseline methods, we use the configurations reported in their original papers when applicable; otherwise, we sweep over a set of hyperparameter values and select the configuration that provides the best accuracy–compute tradeoff under our evaluation setup. We apply the same selection procedure to CoDE-Stop. We further analyze the sensitivity of our method to its hyperparameters in Section 4.3, showing that CoDE-Stop exhibits a smooth and well-behaved tradeoff, making it easy to tune in practice. To ensure transparency, all hyperparameter settings for both our method and the baselines are provided in the appendix.

Metrics. We report four metrics. **Acc** denotes accuracy averaged over all trajectories. **Tok** denotes the average reasoning length (in tokens) per trajectory. **CR** denotes the compression rate relative to the vanilla setting, following Yang et al. (2025). **Cost** denotes the average total token usage per trajectory, including both reasoning tokens and the overhead from intermediate answer generation.

3.2 Results

Figure 5 shows the accuracy–compute tradeoff across four benchmarks and four models. The x-axis reports the total number of tokens generated (including overhead), and the y-axis shows accuracy. Each point corresponds to a method under its best-performing configuration. Across all models and datasets, CoDE-Stop consistently achieves a more favorable accuracy–compute tradeoff than baseline methods. The exact numerical results corresponding to these plots are reported in Table 1, and we provide some qualitative examples in Figure 10 and 11.

Compared to DEER, the most closely related baseline, CoDE-Stop consistently achieves a better accuracy–compute tradeoff. While DEER relies on a fixed confidence threshold and cannot terminate long, unproductive incorrect trajectories, our method additionally captures persistent instability, enabling earlier stopping of such cases.

We further evaluate CoDE-Stop under different prompting strategies on Qwen3-4B. Figure 6 shows the accuracy–compute tradeoff for Vanilla, Budget-Force, Chain-of-Draft, and No-Thinking prompts. CoDE-Stop often improves the tradeoff across these settings by reducing token usage while maintaining or improving accuracy. Notably, even when applied to efficient prompting strategies such as Chain-of-Draft, our method provides additional gains, demonstrating that CoDE-Stop is complementary to prompt-based efficiency techniques and can further reduce unnecessary reasoning across diverse prompting paradigms.

4 Ablation Studies

4.1 Length as a Proxy for Unproductive Reasoning

We study whether trajectory length alone is sufficient to identify and stop unproductive reasoning. As shown in Figure 3, incorrect trajectories tend to be significantly longer than correct ones, suggesting that length may serve as a useful signal for early stopping.

To evaluate this, we compare three stopping strategies in Figure 7, where the x-axis shows the average number of tokens on incorrect trajectories. We first consider DEER, which stops only when confidence exceeds a high threshold and therefore cannot terminate trajectories that remain uncertain yet incorrect. Next, we introduce a simple baseline, *DEER+Fixed-Step*, which stops either when DEER triggers or when the number of reasoning steps exceeds a fixed threshold (we use 40). This baseline leverages trajectory length as a proxy for unproductive reasoning, without using confidence signals on incorrect trajectories.

DEER+Fixed-Step reduces unnecessary computation compared to DEER, confirming that trajectory length is indeed a useful signal for detecting unproductive reasoning. However, CoDE-Stop achieves a better accuracy–compute tradeoff, further reducing the length of incorrect trajectories at matched accuracy. This demonstrates that while length provides a coarse signal for identifying unproductive reasoning, incorporating confidence dynamics—through our degeneration score—leads to more effective early stopping.

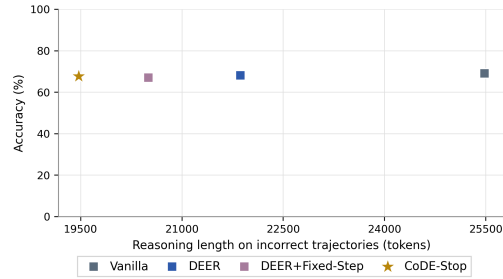


Figure 7: CoDE-Stop reduces unnecessary computation on incorrect rollouts compared to baselines at matched accuracy.

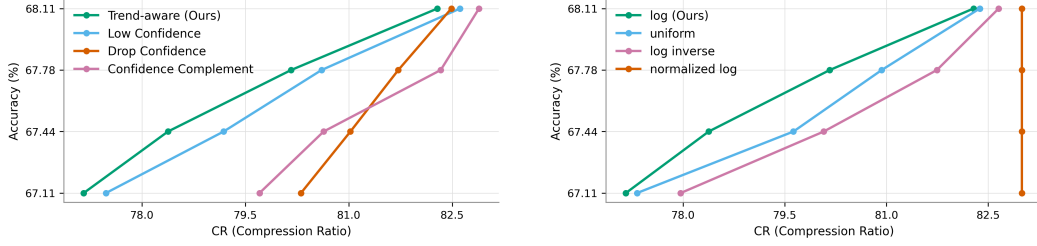
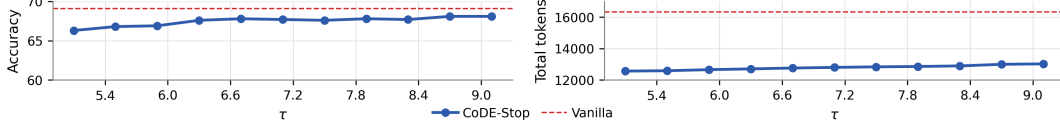
4.2 Ablating degeneration score functions

In this section, we ablate different choices for the v and w functions in the degeneration score defined in Eq. 1, to motivate our design choices.

We first study the degeneration function v , which is designed to assign high values to unconfident reasoning steps and low values to confident ones. We compare four variants: *Confidence Complement* ($1 - c_i$), *Confidence Drop* ($c_{i-1} - c_i$), *Low Confidence* ($\mathbb{1}(c_i < \delta)$), and our *Trend-aware* score ($\mathbb{1}(2c_i - c_{i-1} < \delta)$).

As shown in Figure 8 (Left), our trend-aware score achieves the best accuracy–compression tradeoff across all settings. Methods based on raw confidence (Confidence Complement and Confidence Drop) are less effective due to noise and model-dependent calibration. Compared to Low Confidence, our method performs better because it not only triggers when confidence is low, but also captures sudden drops in confidence, allowing it to better identify unconfident reasoning trajectories.

We next ablate the weighting function w , while fixing v to our trend-aware score. Let T_k denote the token index corresponding to reasoning step k , and define $r = T_K / T_k$, where K is the current reasoning step. We compare four variants: *log weighting* (ours), where $w_k = \log(r) + 1$, *uniform weighting*, where $w_k = 1$, *log inverse weighting*, where $w_k =$

Figure 8: Ablation of degeneration score functions: v_i (left) and w_i (right)Figure 9: Threshold sensitivity: τ controls a smooth accuracy–compute tradeoff.

$\log(1/r) + 1$ and thus emphasizes later steps, and a *normalized log* variant where log weights are normalized to sum to one.

As shown in Figure 8 (Right), log weighting achieves the best accuracy–compression tradeoff, followed by uniform weighting, while log inverse performs the worst. This supports our earlier observation that earlier reasoning steps are more informative for distinguishing correct and incorrect trajectories, and should therefore be weighted more heavily. Finally, we observe that normalizing the weights significantly degrades performance, resulting in near-zero compression gains. This is because trajectory length itself provides a strong signal for identifying unproductive reasoning, as discussed in the previous section, and normalization removes this signal by equalizing contributions across steps.

4.3 Other Ablations

Threshold Sensitivity. We analyze the sensitivity of CoDE-Stop to the degeneration threshold τ . Figure 9 shows results on Qwen3-4B evaluated on the AIME dataset, where we vary τ and report both accuracy and total token usage. We observe a smooth tradeoff between accuracy and compute, indicating that CoDE-Stop is robust to the choice of τ and can be easily tuned to meet different compute budgets.

Reasoning Step Delimiter. We used the token corresponding to “Wait” as the delimiter between reasoning steps and marking the points at which early stopping can occur. This follows prior work that reasoning models tend to include self-reflection words such as “Wait” often during their generation (Yang et al., 2025; Wang et al., 2025). Following Yang et al. (2025), we additionally experiment with using “Alternatively” as a different delimiter. The results are found in Table 7; we observe similar accuracies and efficiency gain as in our main paper, suggesting that CoDE-Stop is robust to a reasonable choice of reasoning step delimiter.

Lower Budget. In all experiments, we set the maximum number of new tokens to 32K. We additionally report results with a lower budget of 16K in Table 2. CoDE-Stop remains effective in this regime, however, the gains are more pronounced at larger budgets, where incorrect trajectories tend to grow substantially longer, increasing the opportunity for degeneration score to contribute to early stopping.

5 Related Works

Prior work has explored improving the efficiency of reasoning in LLMs through training methods, early stopping during inference, and probing internal model states to predict

reasoning correctness. We briefly review these directions and position our approach within the literature.

Efficient Reasoning in Large Language Models. Several works aim to improve reasoning efficiency in large language models through approaches such as concise reasoning training, chain-of-thought compression, or latent reasoning (Arora & Zanette, 2025; Li et al., 2025; Munkhbat et al., 2025; Tan et al., 2025; Aytes et al., 2025; Xu et al., 2025b; Yi et al., 2025). These approaches require additional training or modifications to the model, whereas our method operates at inference time and requires no additional training.

Early Stopping for Reasoning Models. Recent work has proposed several methods for early stopping in reasoning LLMs. Some approaches encourage shorter or truncated reasoning trajectories directly (Xu et al., 2025a; Ma et al., 2025). Other methods terminate reasoning based on signals observed during generation, such as answer convergence (Liu & Wang, 2025; Fu et al., 2024). Several works instead rely on model confidence or uncertainty signals to decide when to stop reasoning (Yang et al., 2025; Yong et al., 2025; Fu et al., 2025; Wang et al., 2025; Wei et al., 2026). Finally, Wang et al. (2026) provide statistical stopping criteria with formal guarantees on reasoning correctness. In contrast, our approach models confidence signals temporally during reasoning and achieves superior early stopping performance across multiple benchmarks.

Predicting Reasoning Success in LLMs. Recent work shows that hidden representations of LLMs encode signals about reasoning success. By probing these internal states, several methods can predict the correctness of reasoning trajectories or final answers early (Zhang et al., 2025; Cencerrado et al., 2026; Afzal et al., 2025; Sun et al., 2025), suggesting that such signals could enable early termination of reasoning. However, these approaches typically require training probes or auxiliary predictors, whereas our method directly leverages confidence dynamics during generation without additional training.

6 Conclusion

In this work, we study the confidence dynamics of large reasoning models for early stopping and introduce CoDE-Stop, a method that combines a ramping confidence threshold with a degeneration score to terminate both successful and unproductive reasoning. CoDE-Stop achieves a strong accuracy–compute tradeoff by reducing unnecessary reasoning while maintaining performance. More broadly, our results show that simple heuristics over confidence—derived from model outputs—already provide effective signals for early stopping, indicating that richer information exists in model activations. Learning to extract such signals directly from model representations is a promising direction for future work.

Acknowledgments

This project was supported in part by a grant from an NSF CAREER AWARD 1942230, the ONR PECASE grant N00014-25-1-2378, ARO’s Early Career Program Award 310902-00001, Army Grant No. W911NF2120076, the NSF award CCF2212458, NSF Award No. 2229885 (NSF Institute for Trustworthy AI in Law and Society, TRAILS), a MURI grant 14262683, DARPA AIQ grant HR00112590066 and an award from meta 314593-00001.

References

- 2024,2025. Aime problems and solutions. https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions, 2024,2025. Accessed: 2026-03.
- Anum Afzal, Florian Matthes, Gal Chechik, and Yftah Ziser. Knowing before saying: Llm representations encode information about chain-of-thought success before completion, 2025. URL <https://arxiv.org/abs/2505.24362>.
- Daman Arora and Andrea Zanette. Training language models to reason efficiently. 2025. URL <https://arxiv.org/abs/2502.04463>.

- Simon A. Aytes, Jinheon Baek, and Sung Ju Hwang. Sketch-of-thought: Efficient llm reasoning with adaptive cognitive-inspired sketching, 2025. URL <https://arxiv.org/abs/2503.05179>.
- Iván Vicente Moreno Cencerrado, Arnau Padrés Masdemont, Anton Gonzalvez Hawthorne, David Demitri Africa, and Lorenzo Pacchiardi. No answer needed: Predicting llm answer accuracy from question-only linear probes, 2026. URL <https://arxiv.org/abs/2509.10625>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- Bercovich et al. Llama-nemotron: Efficient reasoning models, 2025. URL <https://arxiv.org/abs/2505.00949>.
- Yichao Fu, Junda Chen, Siqi Zhu, Zheyu Fu, Zhongdongming Dai, Aurick Qiao, and Hao Zhang. Efficiently serving llm reasoning programs with certaintindex. *arXiv preprint arXiv:2412.20993*, 2024.
- Yichao Fu, Xuewei Wang, Yuandong Tian, and Jiawei Zhao. Deep think with confidence, 2025. URL <https://arxiv.org/abs/2508.15260>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *NeurIPS*, 2021.
- Zeju Li, Jianyuan Zhong, Ziyang Zheng, Xiangyu Wen, Zhijian Xu, Yingying Cheng, Fan Zhang, and Qiang Xu. Compressing chain-of-thought in llms via step entropy, 2025. URL <https://arxiv.org/abs/2508.03346>.
- Xin Liu and Lu Wang. Answer convergence as a signal for early stopping in reasoning. *arXiv preprint arXiv:2506.02536*, 2025.
- Wenjie Ma, Jingxuan He, Charlie Snell, Tyler Griggs, Sewon Min, and Matei Zaharia. Reasoning models can be effective without thinking, 2025. URL <https://arxiv.org/abs/2504.09858>.
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling, 2025. URL <https://arxiv.org/abs/2501.19393>.
- Tergel Munkhbat, Namgyu Ho, Seohyun Kim, Yongjin Yang, Yujin Kim, and Se-Young Yun. Self-training elicits concise reasoning in large language models. *arXiv preprint arXiv:2502.20122*, 2025.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=Ti67584b98>.
- Yucheng Sun, Alessandro Stolfo, and Mrinmaya Sachan. Probing for arithmetic errors in language models. 2025. URL <https://doi.org/10.48550/arXiv.2507.12379>.
- Wenhui Tan, Jiaze Li, Jianzhong Ju, Zhenbo Luo, Ruihua Song, and Jian Luan. Think silently, think fast: Dynamic latent compression of llm reasoning chains. *arXiv preprint arXiv:2505.16552*, 2025.
- Qwen Team. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.

Xi Wang, James McInerney, Lequn Wang, and Nathan Kallus. Entropy after $\langle /Think \rangle$ for reasoning model early exiting. *arXiv preprint arXiv:2509.26522*, 2025.

Xi Wang, Anushri Suresh, Alvin Zhang, Rishi More, William Jurayj, Benjamin Van Durme, Mehrdad Farajtabar, Daniel Khashabi, and Eric Nalisnick. Conformal thinking: Risk control for reasoning on a compute budget, 2026. URL <https://arxiv.org/abs/2602.03814>.

Zihao Wei, Liang Pang, Jiahao Liu, Wenjie Shi, Jingcheng Deng, Shicheng Xu, Zenghao Duan, Fei Sun, Huawei Shen, and Xueqi Cheng. The evolution of thought: Tracking llm overthinking via reasoning dynamics analysis, 2026. URL <https://arxiv.org/abs/2508.17627>.

Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. Chain of draft: Thinking faster by writing less. *arXiv preprint arXiv:2502.18600*, 2025a.

Yige Xu, Xu Guo, Zhiwei Zeng, and Chunyan Miao. SoftCoT: Soft chain-of-thought for efficient reasoning with llms. In *Proceedings of ACL*, 2025b.

Chenxu Yang, Qingyi Si, Yongjie Duan, Zheliang Zhu, Chenyu Zhu, Zheng Lin, Li Cao, and Weiping Wang. Dynamic early exit in reasoning models. *arXiv preprint arXiv:2504.15895*, 2025.

Jingyang Yi, Jiazhen Wang, and Sida Li. Shorterbetter: Guiding reasoning models to find optimal inference length for efficient reasoning. *arXiv preprint arXiv:2504.21370*, 2025.

Xixian Yong, Xiao Zhou, Yingying Zhang, Jinlin Li, Yefeng Zheng, and Xian Wu. Think or not? exploring thinking efficiency in large reasoning models via an information-theoretic lens, 2025. URL <https://arxiv.org/abs/2505.18237>.

Anqi Zhang, Yulin Chen, Jane Pan, Chen Zhao, Aurojit Panda, Jinyang Li, and He He. Reasoning models know when they’re right: Probing hidden states for self-verification, 2025. URL <https://arxiv.org/abs/2504.05419>.

A Prompts

We provide the prompts used in our experiments. The *Vanilla* prompt is the default prompt used in our main experiments. The *Budget Force* prompt explicitly specifies a token budget (set according to the maximum number of new tokens), following prior work (Yang et al., 2025; Muennighoff et al., 2025). For *Chain-of-Draft* (CoD), we follow the prompting strategy in (Xu et al., 2025a). The *No Thinking* prompt is adopted from (Ma et al., 2025).

Vanilla = Please reason step by step, and put your final answer within `\boxed{\}`.

Budget Force = IMPORTANT: You may think for up to [B] tokens. Ensure your response remains within the token limit. Put your final answer in `\boxed{\}`.

Model	Method	AIME24,25				MATH500				GSM8K				GPQA-Diamond				Overall			
		Acc $\uparrow$	Tok $\downarrow$	CR $\downarrow$	Cost $\downarrow$	Acc $\uparrow$	Tok $\downarrow$	CR $\downarrow$	Cost $\downarrow$	Acc $\uparrow$	Tok $\downarrow$	CR $\downarrow$	Cost $\downarrow$	Acc $\uparrow$	Tok $\downarrow$	CR $\downarrow$	Cost $\downarrow$	Acc $\uparrow$	Tok $\downarrow$	CR $\downarrow$	Cost $\downarrow$
Qwen3-4B	Vanilla	69.1	16326	100.0	16326	94.0	5210	100.0	5210	94.8	2306	100.0	2306	55.8	9536	100.0	9536	78.4	8344	100.0	8344
	Think or Not	56.7	14130	86.5	27343	80.8	3768	72.3	6497	87.4	1618	70.2	2462	54.1	6865	72.0	10371	69.8	6595	75.3	11668
	DEER	68.1	13115	80.3	13400	93.7	3803	73.0	3878	94.8	1374	59.6	1400	53.0	6412	67.2	6496	77.4	6176	70.0	6294
	EAT	61.7	13995	85.7	14489	89.0	3642	69.9	3789	94.6	2092	90.7	2161	49.0	6459	67.7	6670	73.6	6547	78.5	6777
	RCPD	68.7	16325	100.0	16960	91.9	4994	95.9	5211	94.7	2140	92.8	2264	55.8	9424	98.8	9825	77.8	8221	96.9	8565
	Answer Convergence CoDE-Stop (Ours)	28.9	5958	36.5	9073	61.5	897	17.2	1656	75.1	401	17.4	825	39.1	947	9.9	1210	51.1	2051	20.3	3191
Qwen3-14B	Vanilla	76.0	15137	100.0	15137	95.3	4878	100.0	4878	96.7	1668	100.0	1668	65.2	8447	100.0	8447	83.3	7532	100.0	7532
	DEER	71.3	11086	73.2	11295	94.8	2808	57.6	2862	96.4	763	45.7	778	56.0	3763	44.5	3799	79.6	4605	55.3	4684
	EAT	66.1	12666	83.7	13130	93.0	3730	76.5	3871	96.7	1591	95.4	1639	58.3	6355	75.2	6533	78.5	6086	82.7	6293
	RCPD	76.0	14526	96.0	15096	88.2	4379	89.8	4573	96.7	1394	83.6	1476	65.2	8098	95.9	8433	81.5	7099	91.3	7394
	Answer Convergence CoDE-Stop (Ours)	–	–	–	–	–	–	–	–	90.1	385	23.1	811	44.7	830	9.8	1060	–	–	–	–
		70.2	10720	70.8	10921	93.0	2481	50.9	2529	95.2	695	41.7	709	53.8	3128	37.0	3159	78.0	4256	50.1	4330
DeepSeek-R1	Vanilla	37.6	14461	100.0	14461	88.4	4576	100.0	4576	90.7	1840	100.0	1840	47.4	9177	100.0	9177	66.0	7514	100.0	7514
	Think or Not	–	–	–	–	87.2	4300	94.0	8391	90.6	1808	98.3	3319	47.5	9056	98.7	18243	–	–	–	–
	DEER	35.0	11508	79.6	12616	85.2	2626	57.4	2762	89.5	1254	68.2	1322	47.0	8032	87.5	8468	64.2	5855	73.2	6292
	EAT	34.3	12624	87.3	13472	79.4	3381	73.9	3577	90.7	1789	97.2	1870	45.7	7948	86.6	8375	62.5	6436	86.3	6824
	RCPD	37.3	14478	100.1	15003	88.6	4576	100.0	4745	90.8	1840	100.0	1934	47.4	9178	100.0	9527	66.0	7518	100.0	7802
	Answer Convergence CoDE-Stop (Ours)	23.4	6328	43.8	8934	67.2	1460	31.9	2526	80.2	518	28.2	827	33.3	2629	28.6	4394	51.0	2734	33.1	4170
Nemotron-SB	Vanilla	55.0	11130	100.0	11130	92.8	3258	100.0	3258	91.7	1211	100.0	1211	53.2	7318	100.0	7318	73.2	5729	100.0	5729
	Think or Not	52.0	9394	84.4	18991	90.3	2754	84.5	5034	91.6	1162	96.0	1724	52.8	6804	93.0	11142	71.7	5028	89.5	9223
	DEER	52.7	9631	86.5	9847	92.4	2448	75.1	2501	91.1	984	81.3	1007	51.7	5883	80.4	5967	72.0	4736	80.8	4830
	EAT	53.7	10514	94.5	11018	87.7	2765	84.9	2886	91.6	1194	98.6	1229	49.0	6334	86.6	6535	70.5	5202	91.1	5417
	RCPD	54.9	10729	96.4	11169	87.5	2992	91.8	3126	91.7	1182	97.6	1253	53.2	7243	99.0	7558	71.8	5536	96.2	5776
	Answer Convergence CoDE-Stop (Ours)	31.9	4341	39.0	9939	66.4	950	29.2	1335	82.6	421	34.8	666	33.9	1231	16.8	1656	53.7	1736	29.9	2399
		52.4	9282	83.4	9493	91.4	2287	70.2	2338	90.8	860	71.0	881	50.9	5634	77.0	5715	71.4	4516	75.4	4607

Table 1: Main results comparing CoDE-Stop with baseline methods across models and benchmarks. Acc denotes accuracy, Tok denotes average reasoning tokens, CR denotes compression rate, and Cost denotes total token compute.

Chain-of-Draft = Please reason step by step, but only keep a minimal draft for each step (at most 10 words). Put your final answer within `\boxed{}`.

Example 1:

Question: There are 15 trees in the grove. Grove workers will plant trees in the grove today.

After they are done, there will be 21 trees. How many trees did the grove workers plant today?

Answer: $21 - 15 = 6$. `\boxed{6}`

Example 2:

Question: If there are 3 cars in the parking lot and 2 more cars arrive, how many cars are in the parking lot?

Answer: $3 + 2 = 5$. `\boxed{5}`

Now answer the following question.

No Thinking = Please reason step by step, and put your final answer within `\boxed{}`. Okay, I think I have finished thinking. `</think>`

B Qual

We provide two qualitative examples of early stopping behavior in Figures 10 and 11.

C Results

We provide a detailed breakdown of the performance of CoDE-Stop and various baselines in Table 1.

D Hyperparameters

Hyperparameters for EAT. Following the setup in the EAT paper Wang et al. (2025), we set the timescale variable, which controls the EMA smoothing in the updates, to 10. We

X Incorrect Steps: 35 savings: 22488 tok length: 32079 tok					
QUESTION Each vertex of a regular octagon is independently colored either red or blue with equal probability. The probability that the octagon can then be rotated so that all of the blue vertices end up at positions where there were originally red vertices is $\frac{m}{n}$, where m and n are relatively prime positive integers. What is $m+n$? Gold answer: 371					
#	CONF	D	INTERMEDIATE ANSWER	REASONING TRACE	
S1	0.816	0.00	~{135}. **Step-by-St...	Wait	<think> Okay, so I need to find the probability that, after coloring each vertex of a regula...
S2	0.621	1.00	~{13}. Wait, no, tha...	Wait	, no, not exactly. Because if you rotate the octagon, then the positions of the blue vertice...
S3	0.832	2.48	~{129}. **Step-by-St...	Wait	, maybe not. Let me think. Alternatively, if we consider that after rotation, the blue verti...
S4	0.820	2.67	~{129}. **Step-by-St...	Wait	, maybe not. Let me try to formalize this. Let me denote the original set of red vertices as...
S5	0.680	3.74	~{129}. **Note:** Th...	Wait	, no, that's not quite.
S6	0.758	4.06	~{257}. Wait, no, th...	Wait	, the problem says "the blue vertices end up at positions where there were originally red ve...
S7	0.695	4.23	~{129}. **Note:** Th...	Wait	, but since the blue positions are the complement of the red positions, that would mean that...
S8	0.695	4.28	~{129}. **Note:** Th...	Wait	, perhaps I need to think in terms of the rotation. Suppose that after rotating the octagon ...
S9	0.676	4.49	~{129}. **Note:** Th...	Wait	, no.
S10	0.609	5.74	~{121}. Wait, no, th...	Wait	, after rotating the octagon by k positions, the color at position i is $c_{(i - k) \bmod 8}$. --
CoDe-Stop — model halts here					
REMAINING TRACE (NOT EXECUTED UNDER EARLY-STOP)					
S11	0.652	5.78	~{121}. Wait, no, th...	Wait	, maybe not. Let me think again. Alternatively, since R is the complement of B , then R_k is ...
S12	0.645	5.81	~{129}. **Note:** Th...	Wait	, but since the rotation by k is a permutation of the octagon, the orbits of the vertices un...
S13	0.578	7.76	~{121}. Wait, no, th...	Wait	, no, not necessarily invariant. But they have some structure. Alternatively, maybe think of...
S14	0.605	7.84	~{121}. Wait, no, th...	Wait	, but the condition is that the blue vertices are in distinct orbits under the rotation. For...
S15	0.730	7.98	~{16} for the rotati...	Wait	, but the problem states that we need that there exists some rotation such that all blue ver...
S16	0.734	8.00	~{16} for each cycle...	Wait	, no.
S17	0.605	9.08	~{121}. Wait, no, th...	Wait	, the problem says that after rotating the octagon, the blue vertices end up at positions wh...
S18	0.504	10.11	~{123}. **Note:** Th...	Wait	, this is a critical point. Let me re-examine. Earlier, I thought that the condition is that...
S19	0.617	10.18	~{123}. Wait, no, th...	Wait	, let me think again. Suppose that we have a rotation k . For the condition to be satisfied, ...
S20	0.602	10.22	~{123}. Wait, no, th...	Wait	, maybe I need to think differently. Let me take a specific example. Suppose that we have a ...
S21	0.586	10.26	~{123}. Wait, no, th...	Wait	, but according to the previous logic, the number of colorings for rotation k is the number ...
S22	0.609	10.58	~{121}. **Note:** Th...	Wait	, let's take that example. Suppose the blue vertices are at positions 0 and 4. Then, if we r...
S23	0.562	12.37	~{121}. Wait, no, I ...	Wait	, no.
S24	0.629	12.89	~{121}. Wait, no, th...	Wait	, if the blue vertices are at positions 0 and 4, then the red vertices are at positions 1, 2...
S25	0.609	13.21	~{121}. Wait, no, th...	Wait	, that can't be. Therefore, my previous example is wrong.
S26	0.637	13.23	~{121}. Wait, no, th...	Wait	, if we have blue vertices at 0 and 4, then after rotating by 4, the blue vertices are at po...
S27	0.609	13.44	~{121}. Wait, no, th...	Wait	, no.
S28	0.629	13.57	~{121}. Wait, no, th...	Wait	, the blue vertices are at positions 1 and 3. Then, the red vertices are 0, 2, 4, 5, 6, 7. I...
S29	0.566	14.88	~{121}. **Note:** Th...	Wait	, this is confusing. Let me try again. Let me think of a coloring where rotating by some k m...
S30	0.707	15.35	~{16}. Wait, no, thi...	Wait	, but in our example, we have two blue vertices in the same orbit. However, according to the...
S31	0.703	16.12	~{123}. Wait, no, th...	Wait	, in the example, we have blue vertices at 1 and 5, which are in the same orbit under rotati...
S32	0.664	16.95	~{121}. Wait, no, th...	Wait	, no, wait.
S33	0.676	17.38	~{121}. Wait, no, th...	Wait	, $\gcd(3, 8)$ is 1, so the rotation by 3 positions has 1 cycle of length 8. Similarly for $k = ...$
S34	0.934	17.66	~{297}. </think>	Wait	, but for $k = 0$, the rotation is identity, so each vertex is its own cycle. So, for $k = 0$, the nu...
S35	0.898	17.73	~{281}. </think>	Wait	, let's go back. For the identity rotation, the condition is that the blue vertices, when rotated...
Model's final answer: 281					

Figure 10: Qualitative Example 1.

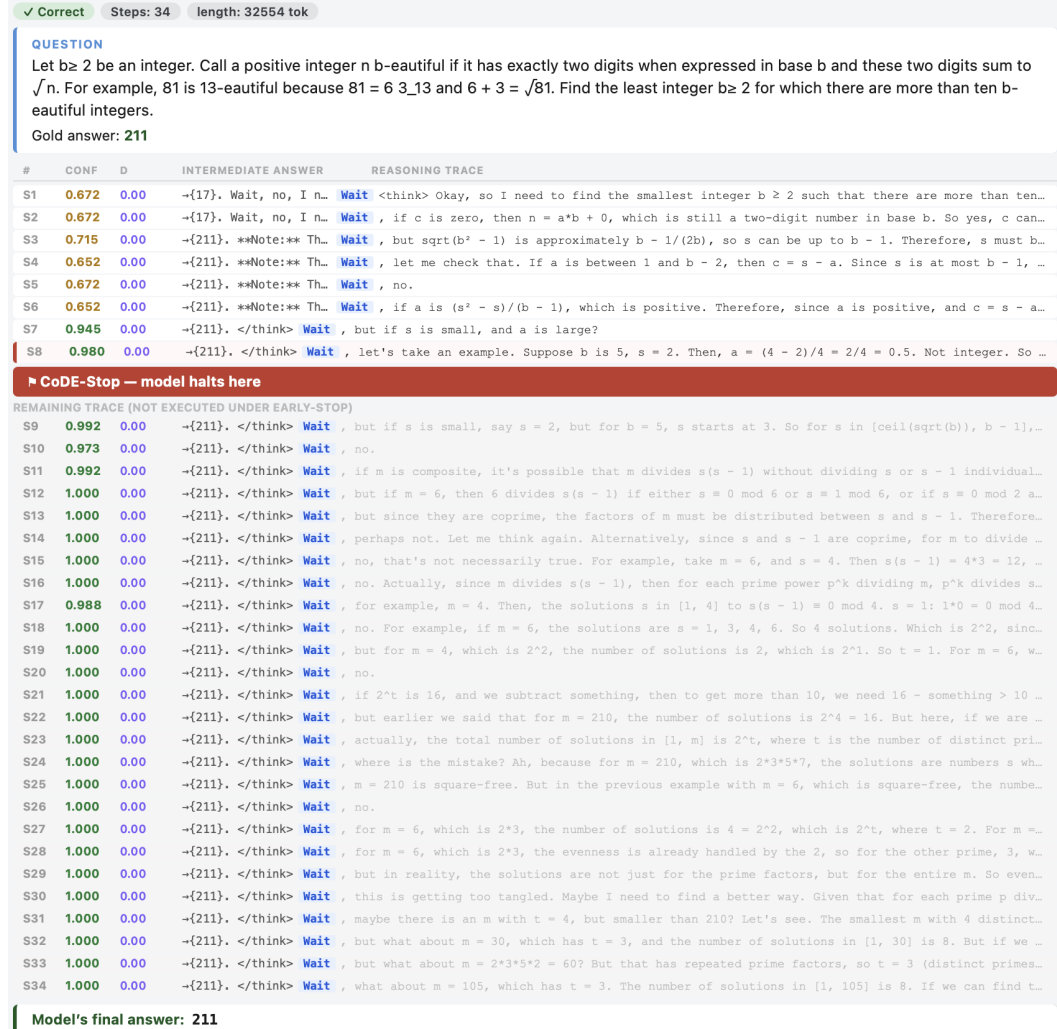


Figure 11: Qualitative Example 2.

Model	Method	AIME24,25				MATH500				GSM8K				GPQA-Diamond				Overall			
		Acc $\uparrow$	Tok $\downarrow$	CR $\downarrow$	Cost $\downarrow$	Acc $\uparrow$	Tok $\downarrow$	CR $\downarrow$	Cost $\downarrow$	Acc $\uparrow$	Tok $\downarrow$	CR $\downarrow$	Cost $\downarrow$	Acc $\uparrow$	Tok $\downarrow$	CR $\downarrow$	Cost $\downarrow$	Acc $\uparrow$	Tok $\downarrow$	CR $\downarrow$	Cost $\downarrow$
Qwen3-4B	Vanilla	62.6	12509	100.0	12509	93.4	4921	100.0	4921	95.2	2236	100.0	2236	55.9	9204	100.0	9204	76.8	7218	100.0	7218
	Think or Not	62.6	12515	100.0	19721	80.3	3490	70.9	5702	88.2	1555	69.5	2374	53.0	6621	71.9	9271	71.0	6045	78.1	9267
	DEER	61.9	10666	85.3	10756	93.2	3666	74.5	3720	95.2	1379	61.7	1405	53.5	6406	69.6	6467	76.0	5529	72.8	5587
	EAT	59.0	11437	91.4	11726	89.8	3635	73.9	3767	94.8	2037	91.1	2103	49.9	6599	71.7	6785	73.4	5927	82.0	6095
	RCPD	58.9	12509	100.0	13011	91.0	4718	95.9	4927	95.2	2073	92.7	2193	55.9	9105	98.9	9503	75.2	7101	96.9	7408
	Answer Convergence CoDE-Stop (Ours)	25.7	4907	39.2	7729	63.1	940	19.1	1727	75.3	401	17.9	825	39.9	961	10.4	1221	51.0	1802	21.7	2876
Qwen3-14B	Vanilla	70.6	12096	100.0	12096	95.1	4600	100.0	4600	96.6	1824	100.0	1824	63.6	8215	100.0	8215	81.5	6684	100.0	6684
	DEER	70.3	9680	80.0	9769	94.4	2744	59.7	2786	96.4	826	45.3	843	56.0	3890	47.4	3924	79.3	4285	0.64	4331
	EAT	65.8	10967	90.7	11268	92.7	3630	78.9	3759	96.9	1716	94.1	1767	57.9	6282	76.5	6451	78.3	5649	85.0	5811
	RCPD	70.9	11615	96.0	12081	88.3	4109	89.3	4292	96.6	1547	84.8	1638	63.7	7816	95.1	8145	79.9	6272	91.3	6539
	Answer Convergence CoDE-Stop (Ours)	69.6	9525	78.7	9612	93.2	2464	53.6	2503	94.7	750	41.1	766	53.4	3196	38.9	3224	77.3	3984	59.6	4026
DeepSeek-R1	Vanilla	33.6	11651	100.0	11651	88.5	4324	100.0	4324	90.5	1812	100.0	1812	46.3	8766	100.0	8766	64.7	6638	100.0	6638
	Think or Not	33.4	11510	98.8	19562	86.9	4140	95.7	7593	90.4	1775	98.0	3252	46.0	8652	98.7	16752	64.2	6519	97.8	11790
	DEER	31.6	10069	86.4	10385	85.5	2615	60.5	2715	90.0	1241	68.5	1296	45.2	7689	87.7	8074	63.1	5404	75.8	5618
	EAT	32.1	10776	92.5	11223	80.3	3388	78.4	3572	89.8	1733	95.6	1812	44.8	7938	90.6	8329	61.8	5959	89.3	6234
	RCPD	33.7	11645	99.9	12067	88.4	4319	99.9	4476	90.4	1812	100.0	1905	46.3	8767	100.0	9101	64.7	6636	100.0	6887
	Answer Convergence CoDE-Stop (Ours)	22.2	5748	49.3	9390	67.7	1424	32.9	2487	80.1	505	27.9	808	32.1	2679	30.6	4779	50.5	2589	35.2	4366
Nemotron-SB	Vanilla	53.4	10441	100.0	10441	93.2	3116	100.0	3116	91.6	1215	100.0	1215	53.3	7313	100.0	7313	72.9	5521	100.0	5521
	Think or Not	50.0	8829	84.6	16075	90.9	2590	83.1	4641	91.6	1161	95.6	1749	53.1	6805	93.1	11022	71.4	4846	89.1	8372
	DEER	51.9	9327	89.3	9476	92.6	2285	73.3	2288	91.1	972	80.0	995	52.1	5779	79.0	5860	71.9	4591	80.4	4665
	EAT	52.8	9937	95.2	10329	88.1	2625	84.2	2736	91.1	1188	97.8	1223	49.2	6258	85.6	6455	70.3	5302	90.7	5186
	RCPD	53.4	10080	96.5	10493	87.3	2847	91.4	2974	91.6	1186	97.6	1257	53.3	7247	99.1	7560	71.4	5040	96.2	5571
	Answer Convergence CoDE-Stop (Ours)	30.2	4168	39.9	5706	67.2	953	30.6	1345	81.9	407	33.5	647	34.1	1181	16.1	1615	53.4	1677	30.0	2328
	CoDE-Stop (Ours)	51.2	9210	88.2	9357	91.5	2018	64.8	2056	90.8	853	70.2	874	51.6	5588	76.4	5667	71.3	4417	74.9	4488

Table 2: Results with B=16K

Model	GSM8K	MATH500	GPQA	AIME
Qwen-4B	(5, 0.0, 0.1)	(5, 0.0, 3.0)	(2, 0.9, 6.5)	(5, 0.0, 7.1)
DeepSeek-R1	(3, 0.9, 23.0)	(1, 0.95, 39.0)	(1, 0.9, 72.0)	(1, 0.9, 90.0)
Nemotron	(3, 0.9, 3.0)	(3, 0.9, 5.5)	(3, 0.9, 10.0)	(1, 0.9, 14.0)
Qwen-14B	(1, 0.5, 1.0)	(5, 0.0, 2.5)	(2, 0.9, 6.0)	(4, 0.9, 10.0)

Table 3: Hyperparameters ($steps, r_{\min}, \tau$) for each model and benchmark. We fix $r_{\max} = 0.95$.

also set the warm-up steps, which ensure that early stopping only occurs after the warm-up stage, to 25, consistent with the original paper. To determine the early stopping threshold δ , we sweep values in $\{10^{-7}, 10^{-5}, 10^{-3}, 10^{-1}\}$ on a subset of AIME dataset with 20 samples. We find that setting the threshold $\delta = 10^{-5}$ yields the best performance (Table 5). We also evaluate different values for the number of rollouts and decoding steps used to estimate the model confidence after the `</think>` token on AIME. Increasing these values provides a less noisy estimate of the model confidence but incurs higher inference cost. We find that setting the number of rollouts to 1 and the number of decoding steps to 2 yields the best results.

Think-or-Not. Yong et al. (2025) is a strong baseline method that utilizes the entropy of trial answers between reasoning steps to determine an appropriate early-stopping point. However, certain features of the implementation make the method more costly than initially presented. For mathematical benchmarks (such as those utilized in our experiment), Think-or-Not generates a list of trial answers using a beam search; the entropy of the top-K answers is then computed (using the token probabilities of each candidate) to estimate the entropy at the chosen point. The beam search process is computationally expensive (requiring the generation of many tokens, and a copy of the KV-cache before trial-answer generation for efficiency). Additionally, this process occurs at every sentence boundary, rather at reasoning-step boundaries as in CoDE-Stop and Yang et al. (2025). This adds significant overhead to this method that is not captured in the reasoning response length reductions reported (that is, Think-or-Not leads to lower reasoning lengths, but potentially high costs). Due to this high computational overhead and our limited resources, we do not evaluate this method on Qwen3-14B.

Additionally, while the manuscript Yong et al. (2025) examines different values of the entropy threshold α for non-mathematical benchmarks, it does not specify the threshold used for the mathematical benchmarks in their main results. In our experiments, we utilized $\alpha = 0.4$, as we found this the best balance between accuracy and efficiency. On Qwen3-4B, we also experimented with the more conservative $\alpha = 0.2$; the results are presented in Table 6.

Model	GSM8K	MATH500	GPQA	AIME
Qwen3-4B	2.5	5.4	12.6	14.4
Qwen3-14B	2.0	5.3	12.5	15.7
DeepSeek-R1-Llama-8B	8.3	22.3	70.4	98.8
Llama-3.1-8B	2.2	4.7	11.9	15.8

Table 4: Average number of reasoning steps across models and benchmarks.

	$\delta = 10^{-1}$	$\delta = 10^{-3}$	$\delta = 10^{-5}$	$\delta = 10^{-7}$
(Acc, Cost)	(20.0, 2037)	(55.0, 9425)	(70.0, 10441)	(70.0, 11381)

Table 5: Performance of EAT on a subset of AIME for different values of δ .

E Reasoning Step Delimiter Ablation Results

We provide a comparison of CoDE-Stop and DEER on different delimiters for reasoning chunks in Table 7.

Method	AIME		MATH500		GSM8K		GPQAD	
	Acc.	Cost	Acc.	Cost	Acc.	Cost	Acc.	Cost
vanilla	68.70%	16329	94.10%	5210	94.80%	2306	55.20%	9638
think_or_not ($\alpha = 0.2$)	68.70%	31706	94.20%	9581	94.80%	3846	55.60%	14924
think_or_not ($\alpha = 0.4$)	56.70%	27343	80.80%	5211	87.40%	2462	54.10%	10371

Table 6: We compare the performance of the Think-or-Not (Yong et al., 2025) baseline with different entropy thresholds (α) with Qwen3-4B. The Vanilla performance values are provided as a reference. We observe that $\alpha = 0.2$ is a very conservative choice; we use $\alpha = 0.4$ in the main experiments.

Delimiter	Method	AIME		MATH500		GSM8K		GPQAD	
		Acc.	Cost	Acc.	Cost	Acc.	Cost	Acc.	Cost
Wait	DEER	68.10%	13400	93.70%	3878	94.80%	1400	53.00%	6496
	CoDE-Stop	67.80%	12759	92.40%	3463	94.50%	1300	52.50%	6283
Altern.	DEER	68.20%	13621	93.70%	3344	94.70%	1173	52.60%	5883
	CoDE-Stop	67.70%	13234	93.20%	3289	94.50%	1133	52.70%	5623

Table 7: Comparison of DEER and CoDE-Stop Methods with different delimiters for reasoning steps. CoDE-Stop maintains similar accuracy and cost reductions compared to this baseline for both potential delimiters, providing evidence for its robustness.